

Betriebssysteme

Probeklausur 2026

Anmerkungen zur Klausur:

- Eigenes Papier und Hilfsmittel außer Kugelschreiber, Füller und Lineal sind **nicht** erlaubt. Es sind weder Bleistifte noch Rotstifte zu nutzen, elektronische Geräte oder zusätzliche Dokumente (z. B. Bücher, Vorlesungs-Skripte) dürfen **nicht** verwendet werden.
- Die Aufgaben müssen auf der Vorderseite der Blätter direkt unter den jeweiligen Angaben bearbeitet werden. Erst wenn die Vorderseiten nicht genügen, dürfen die Vorderseiten der leeren Zusatzblätter am Ende der Klausur genutzt werden. In diesem Fall ist ein Kreuzverweis zwischen Aufgabe und Zusatzblatt nötig.

Viel Erfolg!

Tobias Werner

Nur für die Korrektur:

Aufgabe	1	2	3	Gesamt
a)	5	5	5	
b)	-	5	5	Note
Summe				
Maximum	5	10	10	25

Aufgabe 1: Grundlagen

(5)

a) Betriebssysteme sind ein wesentlicher Bestandteil moderner Rechensysteme.

- Geben Sie die zwei grundsätzlichen Aufgaben von Betriebssystemen in einem Rechner an und nennen Sie jeweils ein Beispiel.

Resourcenverwaltung: zeitliche und räumliche Allokation von Ressourcen.
Beispiel: Wo liegt eine Datei auf einer Festplatte und wann darf welcher Prozess auf diese Datei zugreifen?

Hardware-Abstraktion: Im Sinne einer virtuellen Maschine stellt das OS den Anwendungsprogrammen über Systemaufrufe allgemeine Programmierschnittstellen zur Verfügung, die von konkreten Realisierungen (z. B. einzelnen Dateisystemen, einzelnen Hardware-Herstellern) unabhängig sind.
Beispiel: Der Systemaufruf open() arbeitet unabhängig von konkretem Datenträger (z. B. SSD, Festplatte).

(2P)

- Was versteht man unter einem Micro-Kernel?

Bei Betriebssystemen mit einer Client-Server-Architektur ist das "eigentliche" OS, der Kernel, auf ein Nachrichtenübermittlungssystem zusammengestutzt, welches Prozesse startet, beendet, verwaltet und zwischen diesen Prozessen Nachrichten austauscht. Weitere, typische Kernel-Funktionen (z. B. Dateiverwaltung) sind nicht Teil des Mikro-Kerns, sondern in laufende Anwendungsprozesse ausgelagert.

(1P)

- Was versteht man unter einem Betriebssystem mit Mehrbenutzer-Betrieb?

Das Betriebssystem unterstützt die (in der Regel) gleichzeitige Bedienung durch mehrere, angemeldete Benutzer (z. B. ein Server, auf dem sich mehrere User anmelden und dann jeweils Prozesse starten können).

(1P)

- Kann ein Pfad gleichzeitig kanonisch und relativ sein? Falls ja: Wie sieht ein solcher Pfad aus? Falls nein: Warum nicht?

Das geht: Der Pfad testordner/test.txt ist kanonisch (er enthält keine Pseudo-Verzeichnisse) und relativ (weil er nicht bei der Wurzel C:/ oder / anfängt)

(1P)

Aufgabe 2: Prozessverwaltung

(5+5=10)

a) Die Prozessverwaltung in einem Betriebssystem hält einige Tücken bereit.

- Ein Prozessor hat 4 Rechenkerne. Wie viele Prozesse können auf einem derartigen Prozessor gleichzeitig im Aktiv-Zustand sein?

4 Prozesse können echt-gleichzeitig bearbeitet werden und damit im Zustand Aktiv des Diagramms sein

(1P)

- Was versteht man unter Quasi-Parallelität?

das OS wechselt bzw. verdrängt die aktiven Prozesse in schneller Folge (z. B. im Takt weniger Millisekunden), so dass für den Anwender scheinbar deutlich mehr Prozesse zeitgleich abgearbeitet werden, als der Prozessor das eigentlich könnte

(1P)

- Geben Sie den typischen Grund dafür an, dass ein Prozess in den Blockiert-Zustand wechselt.

Der Prozess hat (i. d. R. per Systemaufruf) auf eine Rechenresource außer der CPU zugegriffen (z. B. Datenträger, Netzwerk), wobei die angeforderten Daten noch nicht im Hauptspeicher zur Verfügung stehen. Damit der Prozess die CPU nicht unnötig belegt, bis die Daten im HS angekommen sind, schaltet das OS den Prozess auf Blockiert und nimmt den Prozess vom Kern.

(1P)

- Warum können Prozesse nicht selbst aus dem Blockiert-Zustand erwachen?

Blockierte Prozesse sind niemals auf der CPU aktiv. Demnach können solche Prozesse keine Rechenoperationen durchführen und damit auch nicht prüfen, ob denn die angeforderte Resource inzwischen zur Verfügung steht.

(1P)

- Was versteht man unter einer *Heuristik* zur Planung von Prozessabläufen?

Ein in der Praxis bewährtes, aber nicht beweisbar gutes Verfahren, um zu entscheiden, welcher Prozess auf der CPU rechnen darf (z. B. unter Berücksichtigung von früheren Rechenzeiten oder Prioritäten).

(1P)

- b) In der Finanzbuchhaltung grauezellen arbeiten drei Bürokräfte *Alice* (A), *Bob* (B) und *Cheryl* (C) gleichzeitig an Unternehmensdokumenten. Konkret müssen regelmäßig zwei besonders wichtige Dokumente bearbeitet werden: die *Personalliste* (P) und der *Quartalsbericht* (Q). Beide Dokumente sind Dateien aus einer handelsüblichen Bürosoftware (z. B. Excel-Dateien). Jede Bürokraft muss hin und wieder jedes Dokument bearbeiten.

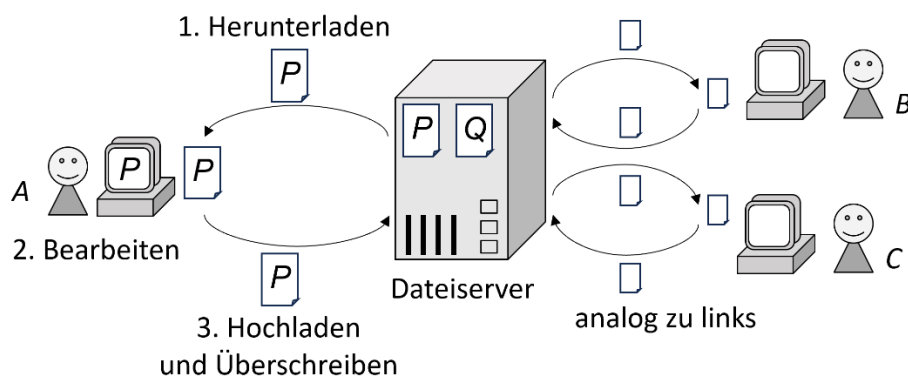
Die Dokumente liegen im aktuellen Stand auf einem Dateiserver, der über das Intranet des Unternehmens zu erreichen ist. Auf dem Dateiserver können die Dokumente aber nicht direkt bearbeitet werden, weil die Bürosoftware dort nicht läuft.

Zur Bearbeitung geht jede Bürokraft daher nach dem Vorgehen V_1 vor:

1. Die Bürokraft verbindet sich mit dem Server und lädt den aktuellen Stand des Dokuments vom Dateiserver auf den eigenen Bürorechner herunter.
2. Die Bürokraft trennt die Verbindung zum Dateiserver und bearbeitet im Anschluss das Dokument lokal auf dem eigenen Rechner.
3. Die Bürokraft verbindet sich zum Server und lädt die fertig bearbeitete Datei dort hoch. Die bearbeitete Datei überschreibt die vorherige Version.

Insgesamt können die Bürokräfte damit zeitgleich die Dateien bearbeiten, z. B. kann A die Datei P bearbeiten, während B die Datei Q bearbeitet.

Der Gesamtzusammenhang ist in folgender Abbildung illustriert:



Hinweise:

- Dass A im Diagramm P bearbeitet, ist nicht zwingend so, sondern nur ein Beispiel. Genauso gut könnte A die Datei Q bearbeiten.
- Da das **reine** Herunterladen, Hochladen und Überschreiben äußerst wenig Zeit braucht (wenige Millisekunden), sei angenommen, dass sich diese Vorgänge **nie** zeitlich überlappen. Es wird z. B. **niemals** zugleich eine Datei heruntergeladen und überschrieben. Gleichzeitiges Bearbeiten ist aber möglich.
- Das Intranet ist zuverlässig genug, dass alle Hochlade- und Herunterlade-Vorgänge auf jeden Fall gelingen. Es gibt keine Verbindungsabbrüche. Der Server ist ebenfalls so zuverlässig, dass alle Dateioperationen immer gelingen.

Im praktischen Betrieb merkt man schnell, dass das obige Vorgehen V_1 ein großes Fehlerpotential birgt. **Erklären Sie** mit Ihrer Kenntnis der Betriebssysteme an einer konkreten Abfolge von Hoch- und Herunterlade-Vorgängen der Bürokräfte A, B und C, **was schief gehen kann**, und **geben Sie ein Fachwort für das Problem an**.

Falls erst A, dann B eine Datei herunterlädt, A bearbeitet und hochlädt, dann B bearbeitet und hochlädt, werden die Änderungen von A überschrieben und gehen verloren. Wenn dagegen die Reihenfolge zufällig passt (d. h. A lädt herunter, bearbeitet, lädt hoch, dann macht B erst im Anschluss selbiges), äußert sich kein Fehler. Das ist eine klassische Race Condition.

(2P)

Um das Problem zu lösen, schlägt die IT ein komplexeres Vorgehen V_2 vor:

1. Sobald ein Dokument heruntergeladen wird, markiert der Dateiserver das Dokument als "in Bearbeitung". Ein so markiertes Dokument kann zunächst von keiner weiteren Bürokraft mehr heruntergeladen werden. Versuche ergeben den Hinweis, zu warten, bis die Bearbeitung abgeschlossen ist.
2. Die Bürokraft, welche das Dokument heruntergeladen hat, bearbeitet **ausschließlich** jenes Dokument am Rechner fertig und lädt das Ergebnis hoch. Dieser Gesamtvorgang solle als Annahme **immer** irgendwann fertig sein.
3. Nach dem Hochladen wird das serverseitige Dokument automatisch ersetzt, die Kopie auf dem Rechner der Bürokraft automatisch gelöscht und zuletzt die Markierung "in Bearbeitung" automatisch entfernt.

Geben Sie **ein Fachwort** aus den Betriebssystemen an, welches diese Realisierung beschreibt. **Löst das Vorgehen V_2 das Problem von zuvor?** Begründen Sie!

Hier ist der Vorschlag ein Mutex: eine Belegungsflagge, so dass nur eine kritische Operation (in dem Fall: der Vorgang vom Herunterladen bis zum Hochladen) zur selben Zeit erfolgen kann. Der Mutex löst in diesem Fall die Race Condition, weil B nicht mehr herunterladen kann, bis A die Bearbeitung abgeschlossen und einen finalen Zustand der Datei hochgeladen hat.

(1P)

Nehmen Sie an, dass das komplexere Vorgehen V_2 der IT wie erklärt umgesetzt wird. Dummerweise zeigt sich jetzt eine andere Sorte Missverhalten:

Angenommen, Bürokräfte können beim Ändern einer beliebigen Datei bemerken, dass sie die andere Datei mitziehen müssen (z. B. um Verweise zu aktualisieren). Die entsprechende Bürokraft lädt in diesem Fall die zweite Datei vom Dateiserver ebenfalls herunter. Erst wenn die Änderungen an **beiden Dateien** abgeschlossen sind, darf die Bürokraft beide Dateien wieder auf den Dateiserver hochladen.

Was geht jetzt – als **Fachbegriff** der Betriebssysteme – schief? **Wie äußert sich das Problem? Erklären Sie** an einer konkreten Folge von Herunter- und Hochlade-Vorgängen der Bürokräfte A, B und C.

Jetzt kann ein Deadlock auftreten: A lädt P herunter und sperrt damit P. Im Anschluss lädt B Q herunter und sperrt damit Q. Falls jetzt A als nächstes Q herunterladen muss, muss sie auf die Freigabe durch B warten, solange kann A auch die Datei P nicht freigeben. Falls jetzt B als nächstes P herunterladen muss, gibt B Q nicht frei, solange er nicht P bekommen kann. Damit hat sich eine endlose Wartesituation eingestellt: die 4 Bedingungen für Deadlocks sind erfüllt, Mutex (s. o.), Zusatzbelegung (A erhöht von nur P auf P und Q, B erhöht von Q auf Q und P), keine externe Freigabe (Datei wird nur hochgeladen und freigegeben, wenn vollständiger Prozess auf beiden Dateien abgeschlossen) und ein Zyklus in den Belegungswünschen ($P \rightarrow Q$ und $Q \rightarrow P$).

(1P)

Nehmen Sie weiterhin die komplexere Realisierung der IT nach V_2 an. Nehmen Sie zudem an, dass die Bearbeitung von Datei P nie den Anlass gibt, auch Datei Q zu bearbeiten. Es kann aber jeder Bürokraft passieren, dass bei der Bearbeitung von Datei Q die Datei P ebenfalls nachgezogen werden muss. In diesem Fall lädt die Bürokraft erst Q, dann P herunter, abschließend beide Dateien gemeinsam hoch. **Kann jetzt etwas schiefgehen?** Begründen Sie mit einer **Bedingung** der VL.

Jetzt gibt es keinen Zyklus in den Belegungswünschen mehr, damit schließt die Bearbeitung der Datei P immer irgendwann ab, so dass im Anschluss auch die Bearbeitung der Datei Q abschließt.

(1P)

Aufgabe 3: Speicherverwaltung

(5+5=10)

a) Bei Systemen mit wenig Hauptspeicher muss man auf Tricks setzen.

- Gegeben sei ein System mit 4 *Speicherrahmen* und 7 *Speicherseiten*, welches mit *FIFO-Verdrängung* arbeitet. Rahmen wie Seiten sind von 0 aufsteigend durchnummeriert. Gegeben sei ferner die folgende Referenzkette:

5, 2, 4, 1, 0, 1, 2, 5, 2, 4, 1, 0

Befüllen Sie die nachstehende Rahmenbelegungstabelle gemäß VL (inklusive Kopfzeilen und -spalten sowie Legende) und markieren Sie *Seitenfehler*, indem Sie neu eingelagerte Seiten unterstreichen.

Hinweis: Es sind ggf. nicht alle Zellen der Tabelle zu füllen! Überzählige Einträge werden mit einem Punktabzug geahndet.

	5	2	4	1	0	1	2	5	2	4	1	0
0	<u>5</u>	5	5	5	<u>0</u>	0	0	0	0	0	<u>1</u>	1
1		<u>2</u>	2	2	2	2	2	<u>5</u>	5	5	5	<u>0</u>
2			<u>4</u>	4	4	4	4	4	<u>2</u>	2	2	2
3				<u>1</u>	1	1	1	1	1	<u>4</u>	4	4

(4P)

- Ein Kommilitone rechnet die obige Aufgabe mit der Belady-Verdrängung durch. Dabei erhält er 11 Seitenfehler. Hat er sich vertan? Begründen Sie!

Hinweis: Sie brauchen hier keinen Algorithmus anwenden.

Belady-Verdrängung ergibt die minimale Seitenfehlerzahl. Ein korrekt durchgeführter Verdrängungsalgorithmus kann nicht weniger Seitenfehler erzeugen. Unser korrekt durchgeführter Algorithmus oben hat aber bereits 10 Seitenfehler erzeugt und damit angeblich Belady unterboten. Weil das nicht möglich ist, muss sich der Kommilitone vertan haben.

(1P)

b) Gegeben sei ein Embedded-System mit virtueller Speicheradressierung.

Virtuelle Adressen seien in dem System 8 Bit große, vorzeichenlose Integer. Jede adressierbare Speicherzelle sei ein Byte groß. Eine Speicherseite umfasse 32 Byte und es gebe 6 Speicherrahmen.

Beantworten Sie in diesem Zusammenhang die folgenden Fragen unter Angabe einer Begründung und gegebenenfalls eines Rechenwegs:

- Wie viele Bytes passen in einen Speicherrahmen?

Rahmen müssen genauso groß sein wie Seiten, damit je eine Seite in exakt einem Rahmen Platz findet. Ein Rahmen hat daher 32 Byte.

(1P)

- Wie viele Speicherseiten gibt es insgesamt im virtuellen Adressraum?

Der virtuelle Adressraum umfasst $2^8 = 256$ virtuelle Adressen. Alle Speicherseiten sind gleich groß, nämlich 32 Byte, damit gibt es im virtuellen Adressraum $256 / 32 = 8$ Speicherseiten.

(1P)

- Wie groß ist der physikalisch vorhandene Hauptspeicher des Systems?

Der physikalisch vorhandene Hauptspeicher wird durch die Rahmen vollständig und exakt abgedeckt. Deshalb gibt es $6 * 32 = 192$ Bytes an Hauptspeicher.

(1P)

- Zerlegen Sie die virtuelle Adresse 11010001 in eine Seitennummer und ein Offset auf der zugehörigen Seite.

Es gibt 8 Speicherseiten. Um diese Speicherseiten durchzuzählen, braucht man 3 Bit (für Werte 0...7, binär 000 bis 111). Die Speicherseite wird von den höchsten Bits gezählt, also ist 110 aus der Adresse die Seitennummer und 10001 das Offset auf der Seite -- es wird also das Byte mit Index 17 auf der Speicherseite mit Index 6 angesprochen -- insgesamt das Byte mit Index 209 relativ zum gesamten virtuellen Adressraum

(1P)

- Angenommen, das OS habe für einen Prozess gerade folgende Seitentabelle:

Seitennummer	Rahmennummer	Present?
000	010	yes
001	011	yes
010	000	no
011	000	yes
100	001	yes
101	000	no
110	101	yes
111	100	yes

Welche physikalische Adresse korrespondiert zur virtuellen Adresse 01001110?

Zu dieser virtuellen Adresse gibt es keine physikalische Adresse, da die zugehörige Seite (010) laut Present-Bit gerade gar nicht in einen Seitenrahmen geladen ist. Die Seite 010 befindet sich stattdessen in ausgelagertem Zustand auf der Festplatte.

(1P)

Leerseite zur freien Verwendung. Bitte hier bearbeitete Aufgaben referenzieren!
(z. B. "Lösung zu 2b", "Fortsetzung zu 3").

Leerseite zur freien Verwendung. Bitte hier bearbeitete Aufgaben referenzieren!
(z. B. "Lösung zu 2b", "Fortsetzung zu 3").